

OUC26夏移动软件开发-实验5

姓名：马一诺 学号：24020007088

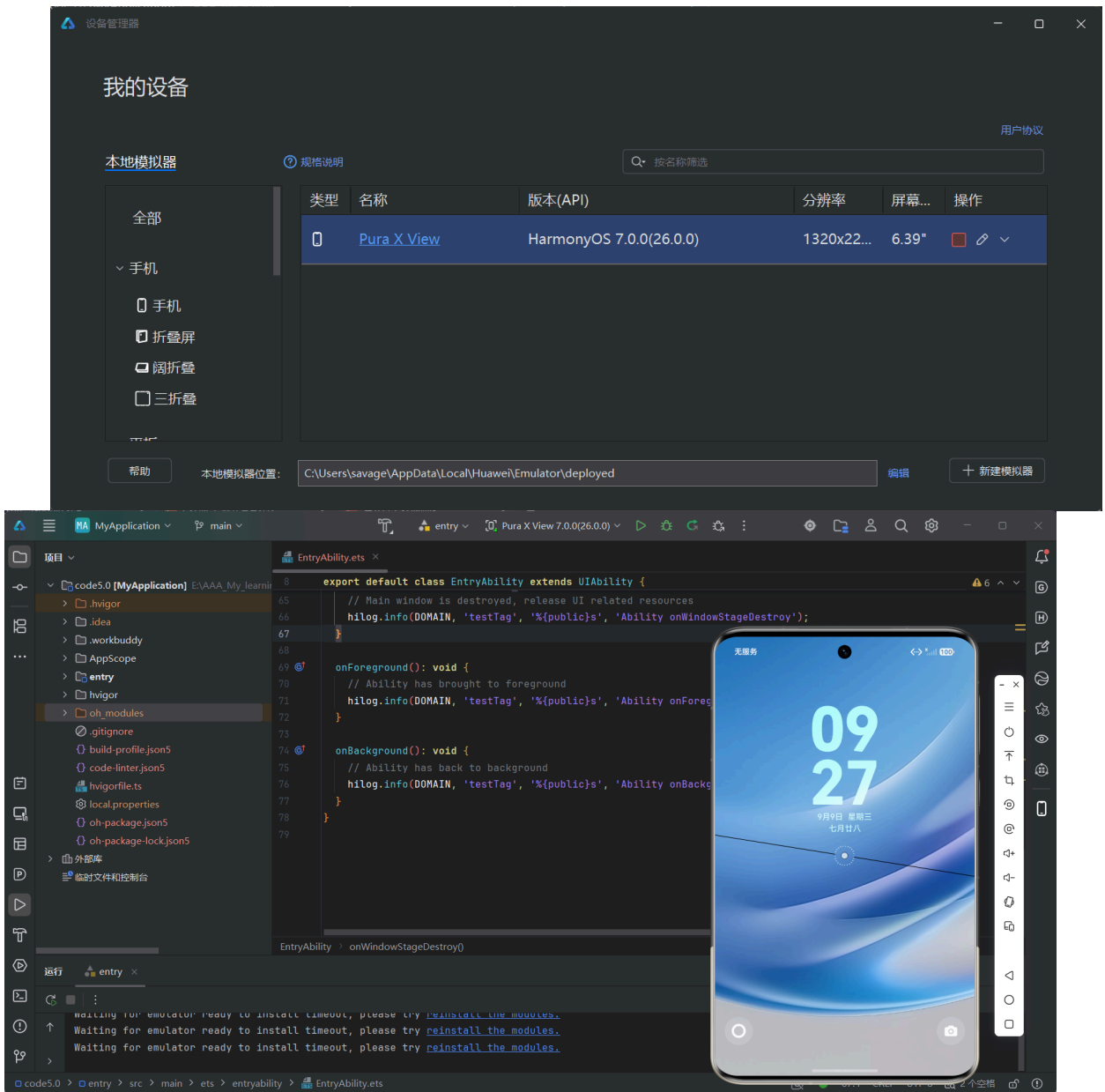
项目	内容
姓名和学号	马一诺，24020007088
所属课程	中国海洋大学26夏《移动软件开发》
实验名称	实验5：HarmonyOS 多功能计算器
博客地址	https://blog.csdn.net/2604_96828618/article/details/164712446?fromshare=blogdetail&sharetype=blogdetail&sharerId=164712446&sharerefer=PC&sharesource=2604_96828618&sharefrom=from_link
代码仓库地址	https://github.com/1-qaz-2-wsx/ouc26-mobile-dev/tree/main/lab5/code5.0

一、实验内容

1. 实验目标与开发环境

本实验使用 HarmonyOS 的 ArkTS / ArkUI 声明式框架，参照华为开发者官方文档《计算器-关键场景示例》实现一个计算器应用，并在此基础上扩展为一个"多功能计算器"。除基础的普通计算器、科学计算器外，还新增了亲戚称呼换算、进制转换、单位换算和汇率换算共六个功能。

开发环境为 DevEco Studio，工程兼容 HarmonyOS SDK 6.1.1(24)，targetSdk 26.0.0。



2. 项目结构与六大功能页面

项目注册了六个同层级页面，每个页面左上角显示功能标题，右上角 三 菜单可切换到其余功能：

页面	路径	主要功能
普通计算器	pages/MainPage	四则运算、百分比、正负号、清除 / 删除
科学计算器	pages/ScientificPage	三角函数、对数、幂、括号、记忆、角度制切换
亲戚称呼换算	pages/RelationshipPage	三种查询模式、性别选择、关系链推演
进制转换	pages/BaseConverterPage	十六进制 / 十进制 / 八进制 / 二进制互转
单位换算	pages/UnitConverterPage	长度 / 重量 / 面积 / 体积 / 温度 / 速度 / 数据
汇率换算	pages/CurrencyConverterPage	8 种货币按静态汇率换算

主要目录如下：

```

1 code5.0/
2   └─ AppScope/
3   └─ entry/src/main/
4     └─ ets/
5       │   └─ entryability/EntryAbility.ets    # 加载主页、全屏窗口、避让区
6       │   └─ entrybackupability/
7       │   └─ pages/                            # 六个功能页面
8       │       └─ MainPage.ets                  # 普通计算器
9       │       └─ ScientificPage.ets            # 科学计算器
10      │       └─ RelationshipPage.ets           # 亲戚称呼换算
11      │       └─ BaseConverterPage.ets         # 进制转换
12      │       └─ UnitConverterPage.ets         # 单位换算
13      │       └─ CurrencyConverterPage.ets     # 汇率换算
14      │   └─ common/
15      │       └─ constants/CommonConstants.ets
16      │       └─ util/
17      │           └─ CalculateUtil.ets          # 表达式解析与求值
18      │           └─ KinshipUtil.ets           # 亲戚关系图引擎
19      │           └─ BaseConverterUtil.ets      # 进制转换
20      │           └─ UnitConverterUtil.ets      # 单位换算（仿射映射）
21      │           └─ CurrencyConverterUtil.ets  # 汇率换算（静态汇率表）
22      │           └─ NavMenu.ets               # 统一功能菜单
23      │           └─ CheckEmptyUtil.ets
24      │           └─ Logger.ets
25      │   └─ viewmodel/                        # 键盘按钮数据模型
26      │       └─ PressKeysItem.ets / PressKeysViewModel.ets
27      │       └─ ScientificKeyItem.ets / ScientificKeyViewModel.ets
28      │       └─ RelationshipKeyItem.ets / RelationshipKeyViewModel.ets
29      └─ resources/base/
30      │   └─ element/                          # string.json / color.json /
31      │       └─ media/                        # 键盘与图标素材
32      │       └─ profile/main_pages.json       # 页面注册表
33      └─ build-profile.json5
34      └─ oh-package.json5

```

这种结构把界面渲染和业务逻辑分开：页面只负责交互和展示，`common/util` 下的工具类承载可复用的纯逻辑，便于测试与维护。



3. 全局配置与 iOS 风格主题

`main_pages.json` 注册了全部六个页面，`module.json5` 指定入口为 `EntryAbility`：

```

1  {
2    "src": [
3      "pages/MainPage",
4      "pages/ScientificPage",
5      "pages/RelationshipPage",
6      "pages/BaseConverterPage",
7      "pages/UnitConverterPage",
8      "pages/CurrencyConverterPage"
9    ]
10 }

```

页面视觉统一为 iOS 计算器风格：浅灰背景 #F2F2F7，数字键白色，运算符 / 删除键 #E5E5EA，等号键橙色 #FF9500，切换 / 激活态使用系统蓝 #007AFF。所有按键使用大圆角（基础键 borderRadius(40)、函数键 borderRadius(35)）呈现胶囊形，且不添加阴影，靠颜色区分层级。配色在 color.json 中集中管理：

```

1  {
2    "color": [
3      { "name": "all_back_color", "value": "#F2F2F7" },
4      { "name": "equals_back_color", "value": "#007DFF" }
5    ]
6  }

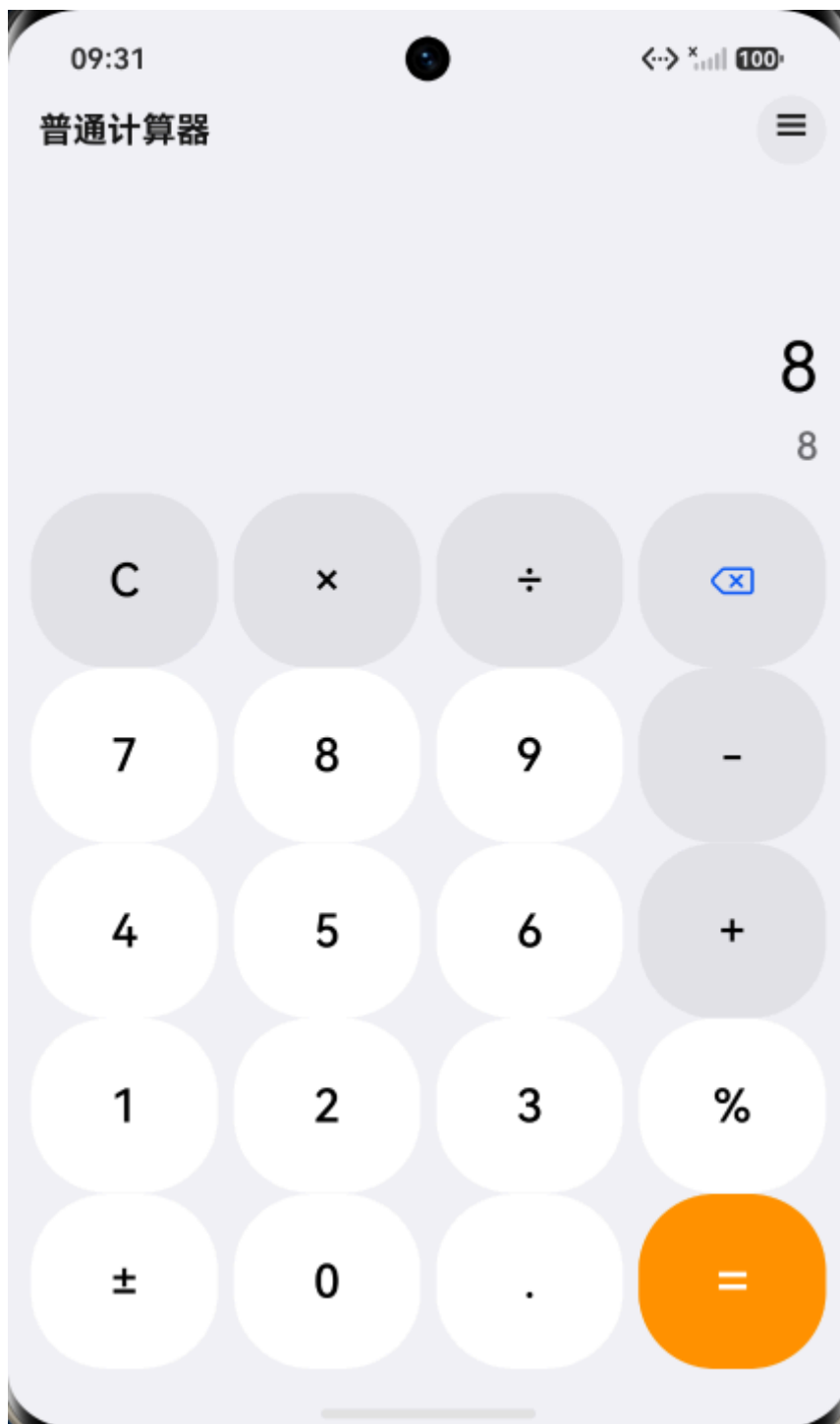
```

EntryAbility.ets 在 onWindowStageCreate 中加载 pages/MainPage，调用 setWindowLayoutFullScreen(true) 开启全屏，并通过 getWindowAvoidArea 获取状态栏和导航条的避让高度，写入 AppStorage（topRectHeight / bottomRectHeight），同时注册 avoidAreaChange 监听以在旋转或分屏时动态更新：

```

1  windowStage.loadContent('pages/MainPage', (err) => { ... });
2
3  windowClass.setWindowLayoutFullScreen(true);
4  const avoidArea =
5    windowClass.getWindowAvoidArea(window.AvoidAreaType.TYPE_SYSTEM);
6  AppStorage.setOrCreate('topRectHeight', avoidArea.topRect.height);

```



4. 统一功能导航菜单

为避免六个页面各自维护一份菜单，项目新增 `common/util/NavMenu.ets`。

`buildNavMenu(current)` 返回 `Array<MenuElement>`，其中当前页面所在项末尾加 ☒ 标记，点击其他项通过 `router.replaceUrl` 跳转：

```
1  const ENTRIES: NavEntry[] = [  
2    { key: 'calculator', label: '普通计算器', url: 'pages/MainPage' },  
3    { key: 'scientific', label: '科学计算器', url: 'pages/ScientificPage' },  
4    { key: 'relationship', label: '亲戚称呼换算', url: 'pages/RelationshipPage'  
    },  
5    { key: 'base', label: '进制转换', url: 'pages/BaseConverterPage' },  
6    { key: 'unit', label: '单位换算', url: 'pages/UnitConverterPage' },  
7    { key: 'currency', label: '汇率换算', url: 'pages/CurrencyConverterPage' }  
8  ];
```

```

9
10 export function buildNavMenu(current: string): Array<MenuElement> {
11   const result: Array<MenuElement> = [];
12   ENTRIES.forEach((entry: NavEntry) => {
13     const label = entry.key === current ? `${entry.label} ✓` : entry.label;
14     result.push({
15       value: label,
16       action: (): void => {
17         if (entry.key !== current) {
18           router.replaceUrl({ url: entry.url });
19         }
20       }
21     });
22   });
23   return result;
24 }

```

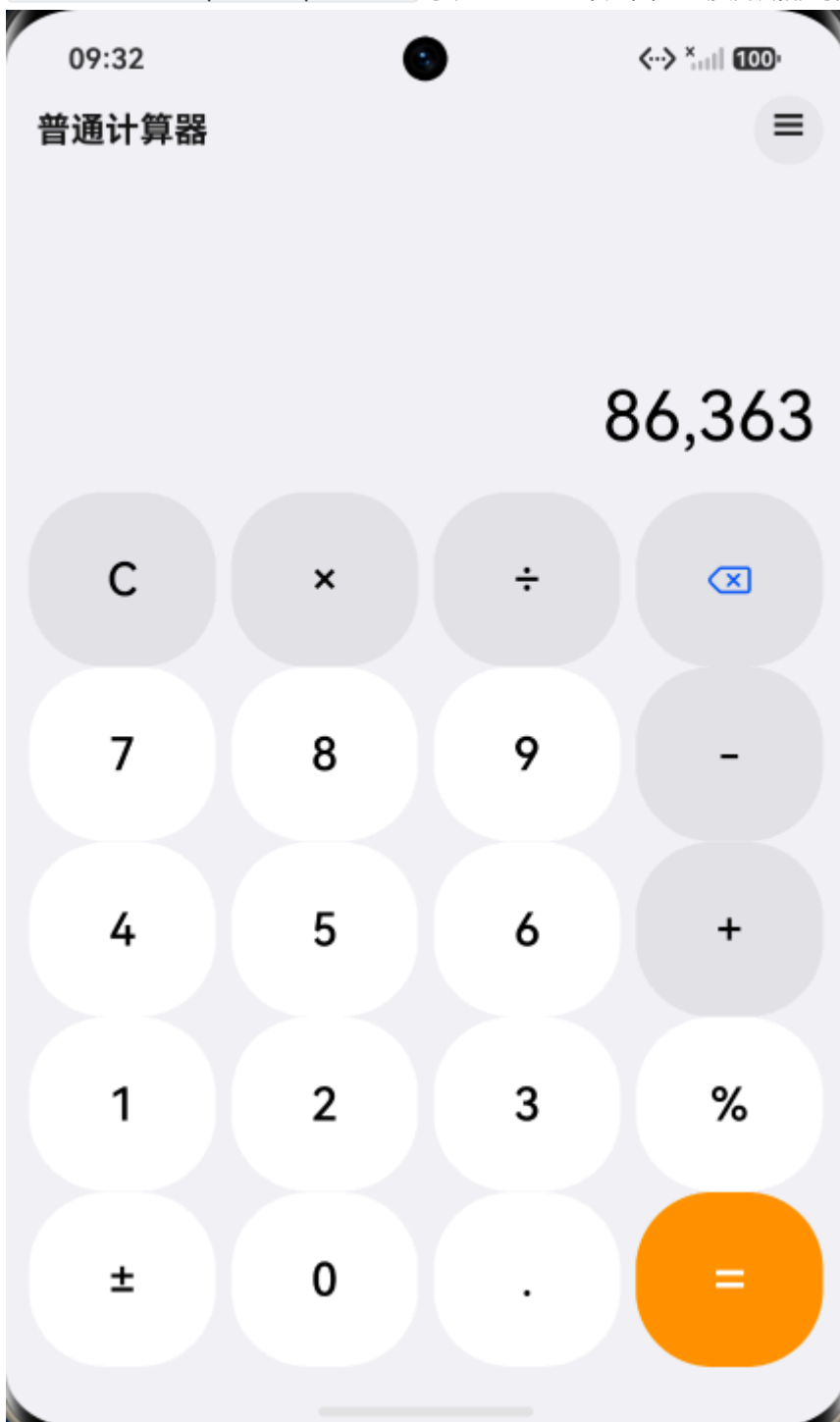
每个页面的标题栏右侧都调用 `bindMenu(buildNavMenu('当前功能'))`，实现"所有页面都显示全部功能、当前功能打勾"的统一交互。使用 `replaceUrl` 而非 `pushUrl`，保证页面栈始终为单层，六个页面处于同一层级、无父子返回关系。



5. 普通计算器键盘与布局

普通计算器采用 4 列 × 5 行键盘，按键通过 `ForEach` 从 `PresskeysViewModel` 的数据集中生成，行列均使用 `layoutweight(1)` 等分，使其自适应不同屏幕宽度。数字键白色、运算符和删除键浅灰、等号键橙色，与参考图保持一致。

输入处理沿用华为官方示例的状态模型：`expressions` 数组保存数字与运算符，`inputNumber` / `inputOperators` / `negateLast` 分别处理数字、符号和正负号，`getResult` 调用 `calculateUtil.parseExpression` 求值。显示区采用固定高度并底部对齐，避免输入前后布局跳动。



6. 科学计算器键盘与状态机

科学计算器在基础键盘之上增加 6 列 × 4 行函数区，包含括号、记忆键（MC / M+ / M- / MR）、阶乘 / 平方 / 立方 / 幂、 π / e、 $1/x$ / $\sqrt{}$ / $\sqrt[3]{}$ / $\sqrt[n]{}$ 、Inv、角度制切换（Rad / Deg）以及 e / $\ln$ / $\lg$ / $\sin$ / $\cos$ / $\tan$ 。

其输入状态机包含以下核心状态：

- `tokens`：已提交的数字、运算符、括号和函数名的扁平序列；
- `currentNum`：正在输入的数字；
- `pendingOp` / `pendingOperand`： $x \wedge y$ 与 $\sqrt[n]{x}$ 两步式运算的暂存；

- `memoryValue`: 记忆值;
- `justApplied`: 函数 / 常量 / 百分比结果后, 下一次输入数字开启新数。

函数键有两种行为: 当前已有数字时立即对单个数求值 (如输入 `30` 后点 `sin`) ; 否则以"函数 + 左括号"形式进入待输入状态, 使参数可以是完整的子表达式 (如 `sin(π/2)`) 。角度制切换会把 `angleMode` 传给求值函数, 三角函数与反三角函数据此做弧度 / 角度换算。

09:33

<--> x 100

科学计算器

Rad



普通计算器

科学计算器 ✓

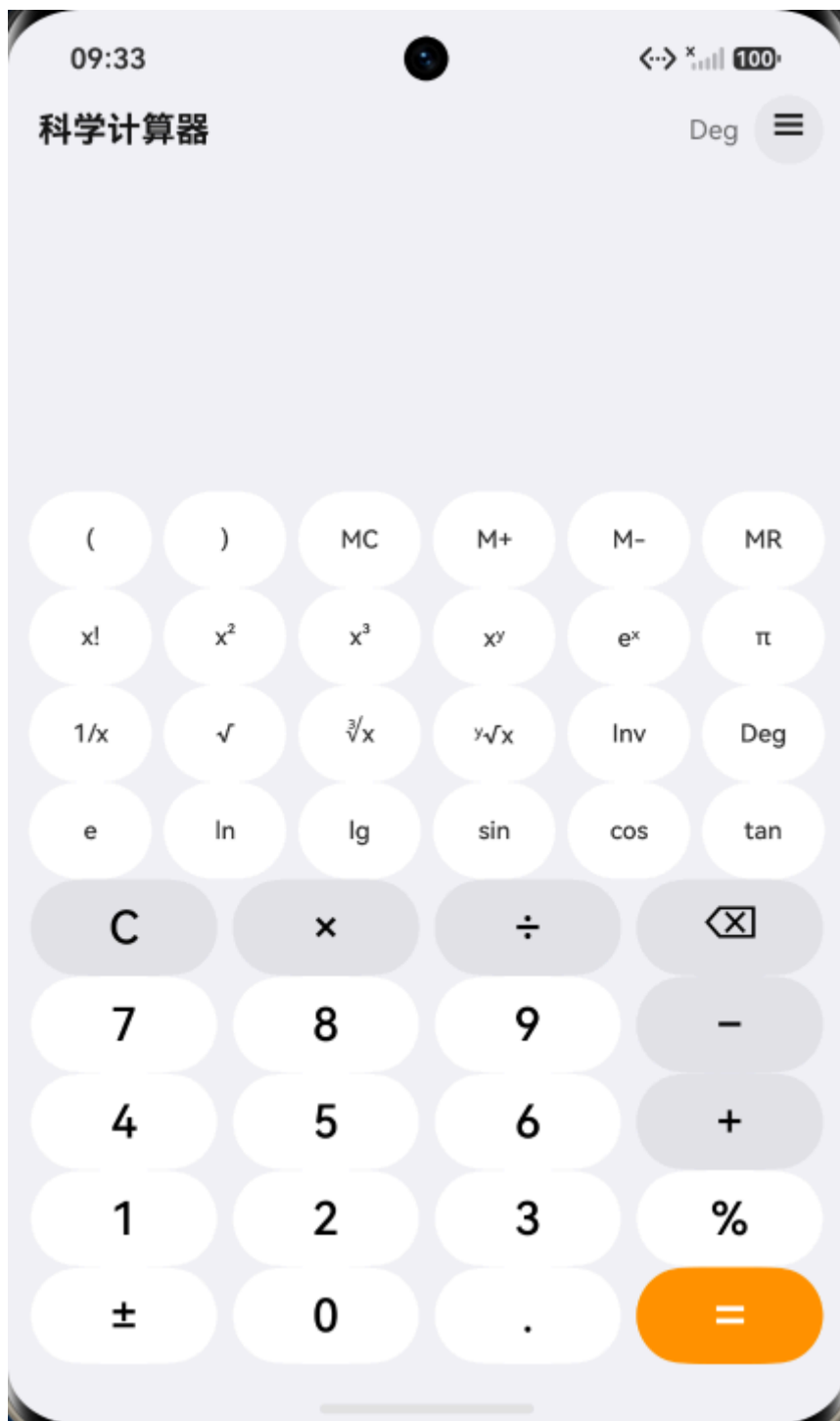
亲戚称呼换算

进制转换

单位换算

汇率换算





7. 表达式解析器（调度场算法 + 函数 + 一元负号）

科学计算器的核心是 `common/util/CalculateUtils`。它先对 `%` 后缀和末尾多余运算符做预处理，再用调度场算法（shunting-yard）把中缀表达式转为后缀队列，最后求值：

```

1 static readonly FUNCTIONS: string[] = [
2     'sin', 'cos', 'tan', 'asin', 'acos', 'atan',
3     'ln', 'log', 'exp', 'pow10', 'sqrt', 'cbrt',
4     'abs', 'square', 'cube', 'fact', 'inv', 'neg'
5 ];
6
7 parseExpression(expressions: Array<string>, angleMode?: string): string {
8     // '(' 入栈; ')' 弹栈直到 '(', 并在 '(' 下方紧邻函数时把函数一起弹到输出队列;
9     // 函数名入栈; 运算符按优先级弹栈; 数字直接进输出队列;
10    // 最后返回 dealQueue(outputQueue, angleMode)。
11 }

```

`dealQueue` 求值时, 遇函数名弹出一个操作数调用 `applyFunction`, 遇二元运算符弹出两个操作数调用 `calResult`。`applyFunction` 支持三角 / 反三角、对数、幂、开方、阶乘等运算, 并根据 `angleMode` 做角度换算:

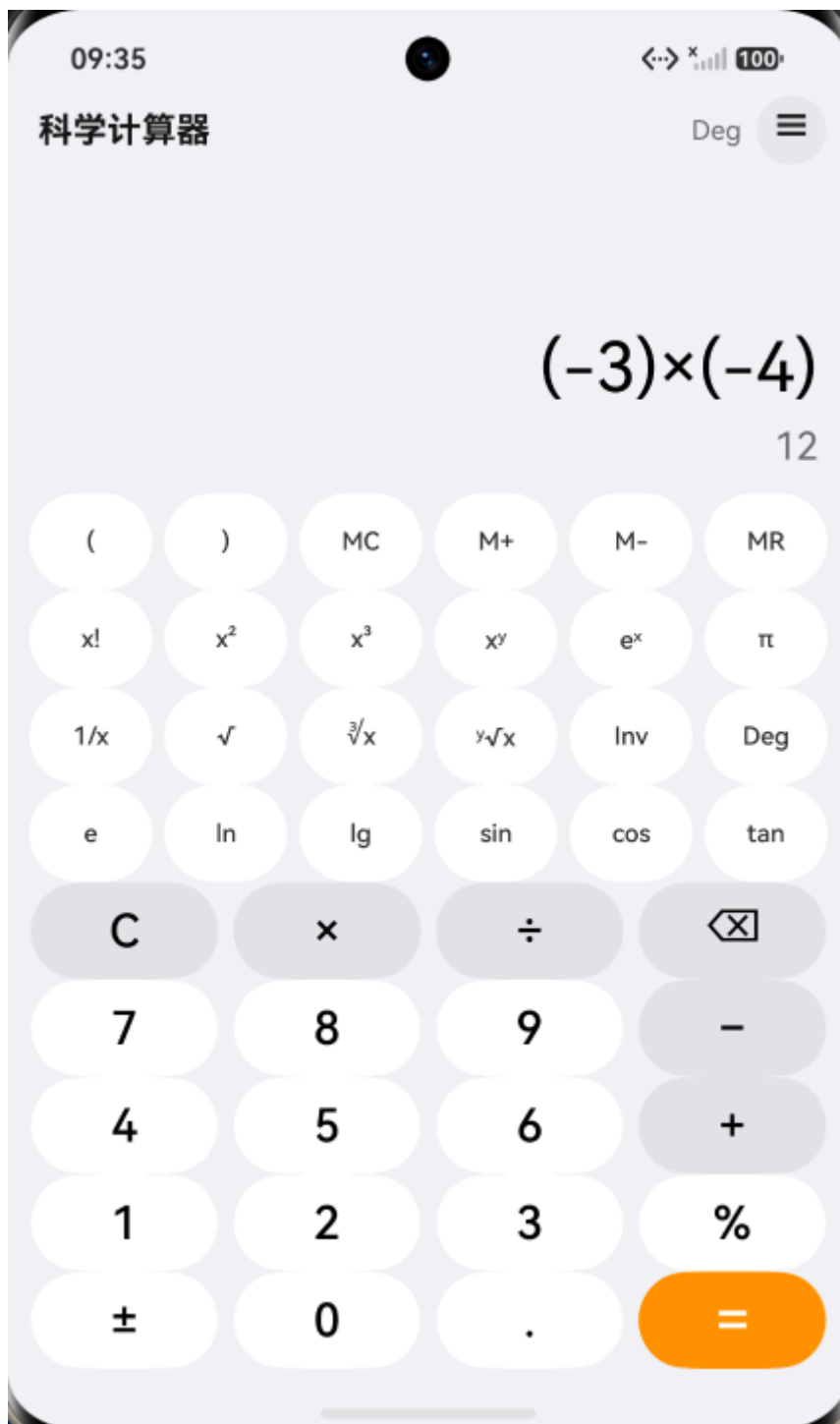
```

1 case 'sin': r = Math.sin(useDeg ? n * Math.PI / 180 : n); break;
2 case 'asin': r = useDeg ? Math.asin(n) * 180 / Math.PI : Math.asin(n); break;

```

针对浮点运算噪声 (例如 $\tan(\pi/4)$ 得到 0.9999999999999983 而非 1), `applyFunction` 与 `applyPow` 在返回前用 `roundSignificant(value, 12)` 按 12 位有效数字舍入, 得到 1、-1 等干净结果。

负数的处理统一为"带符号的数字 token": `-` 键在表达式开头、`(` 之后或运算符之后被识别为一元负号, 进入 `currentNum = '-'` 状态, 后续数字拼接成 `-3`, 因此 `(-3)*(-4)` 可正确得到 12, `sqrt(-4)` 会得到 `NaN` 并显示错误。



8. 亲戚称呼换算（关系图引擎）

亲戚称呼换算通过 `common/util/kinshiputil.ets` 实现，提供三种查询模式：

1. **查询亲戚的称呼**：按"爸爸 / 妈妈 / 老公 / 老婆 / 儿子 / 女儿 / 哥哥 / 弟弟 / 姐姐 / 妹妹"十个关系键依次推演，输出对应的中文称呼；
2. **查询亲戚对我的称呼**：同一关系链反向映射，输出对方对我的称呼；
3. **根据称呼查询关系**：通过 `TITLE_TO_RELATION` 映射表，由称呼反查关系描述。

关系链用一个 `KinshipNode`（代际 `gen`、性别 `gender`、亲系 `side`、长幼 `birthorder`、类别 `category`）表示。每个关系 token 根据当前节点类别按中式亲属规则转移，例如"爸爸的哥哥"得到类别 `parentsibling`、长幼 `elder`，最终映射为"伯父"：

```
1  if (node.gen === 1 && node.category === 'parentsibling') {  
2    if (node.side === 'paternal' && node.gender === 'M' && node.birthOrder ===  
    'elder') return '伯父';  
3    if (node.side === 'paternal' && node.gender === 'M' && node.birthOrder ===  
    'younger') return '叔叔';  
4    // ...  
5  }
```

性别默认选择"女"。为避免"爸爸的老公"这类非法输入显示"—"后无法继续，`tryAppendToken` 会先试算追加 token 后的关系链，只有结果合法才返回新链，否则静默丢弃本次点击，保留原链并允许继续输入其他有效关系。

09:37

<--> x 100

亲戚称呼换算



我的性别



女



男

TA 是您的称呼

舅舅

妈妈的弟弟

切换

查询亲戚的称呼

妈妈

爸爸

DEL

CLR

老婆

老公

女儿

儿子

姐姐

妹妹

哥哥

弟弟

09:37

100%

亲戚称呼换算



我的性别



女



男

TA 称呼您

外甥女

妈妈的弟弟

切换

查询亲戚对我的称呼

妈妈

爸爸

DEL

CLR

老婆

老公

女儿

儿子

姐姐

妹妹

哥哥

弟弟



9. 进制转换

进制转换页提供十六进制 (HEX)、十进制 (DEC)、八进制 (OCT)、二进制 (BIN) 四个进制。`BaseConverterUtil` 负责任意进制互转，先转十进制再转目标进制，并校验每个数字在当前进制下是否合法：

```
1 static toDecimal(input: string, fromBase: number): string {
2     let value = 0;
3     for (let i = 0; i < input.length; i++) {
4         const index =
5             BaseConverterUtil.BASE_DIGITS.indexOf(input[i].toUpperCase());
6         if (index < 0 || index >= fromBase) return '';
7         value = value * fromBase + index;
8     }
9     return value.toString();
10 }
```

```
9  }
10
11  static fromDecimal(decimal: string, toBase: number): string {
12      let num = Number(decimal);
13      let result = '';
14      while (num > 0) {
15          result = BaseConverterUtil.BASE_DIGITS.charAt(num % toBase) + result;
16          num = Math.floor(num / toBase);
17      }
18      return result;
19  }
```

页面上方是 HEX / DEC / OCT / BIN 的切换胶囊，中间显示当前输入及四种进制的同步换算结果，下方是十六进制数字键盘。切换进制时会按当前输入重新解释并转换，非法数字在当前进制下以灰色置灰不可点。

09:37

<--> x 100

进制转换



HEX

DEC

OCT

BIN

13

十进制 DEC

0

0

0

0

十六进制

十进制

八进制

二进制

7

8

9

A

4

5

6

B

1

2

3

C

0

F

E

D



C

09:38

<...> x 100%

进制转换



HEX

DEC

OCT

BIN

D

十六进制 HEX

十六进制

十进制

八进制

二进制

0

0

0

0

7

8

9

A

4

5

6

B

1

2

3

C

0

F

E

D



C

09:38

<...> x 100%

进制转换



HEX

DEC

OCT

BIN

15

八进制 OCT

0

0

0

0

十六进制

十进制

八进制

二进制

7

8

9

A

4

5

6

B

1

2

3

C

0

F

E

D



C



10. 单位换算

单位换算支持长度、重量、面积、体积、温度、速度、数据七类。`UnitConverterUtil` 用"基准单位 + 仿射映射"统一描述： $\text{baseValue} = \text{value} * \text{factor} + \text{offset}$ ，因此温度这类带偏移量的换算也能正确处理：

```
1 export class UnitDef {
2   label: string;
3   factor: number;
4   offset: number;
5 }
6
7 // 温度: 摄氏度 / 华氏度 / 开尔文
8 new UnitDef('摄氏度', 1, 0),
9 new UnitDef('华氏度', 5 / 9, -160 / 9),
10 new UnitDef('开尔文', 1, -273.15)
```

`convert(value, from, to)` 先把输入换算到基准单位，再从基准单位换算到目标单位。页面上方是分类横向滚动条，中间选择"从 / 到"单位，结果显示实时换算值。



11. 汇率换算

汇率换算内置人民币、美元、欧元、日元、英镑、港币、韩元、澳元 8 种货币，`CurrencyConverterUtil` 维护一张以人民币为基准的静态汇率表：

```
1 static getCurrencies(): CurrencyDef[] {
2     return [
3         new CurrencyDef('CNY', '人民币', 1),
4         new CurrencyDef('USD', '美元', 7.20),
5         new CurrencyDef('EUR', '欧元', 7.80),
6         // ...
7     ];
8 }
9
10 static convert(value: number, from: CurrencyDef, to: CurrencyDef): number {
11     return value * from.rate / to.rate;
12 }
```

页面上方选择"从 / 到"货币，下方显示换算结果，并标注"汇率仅供参考（静态）"。真实应用中可替换为动态汇率接口。



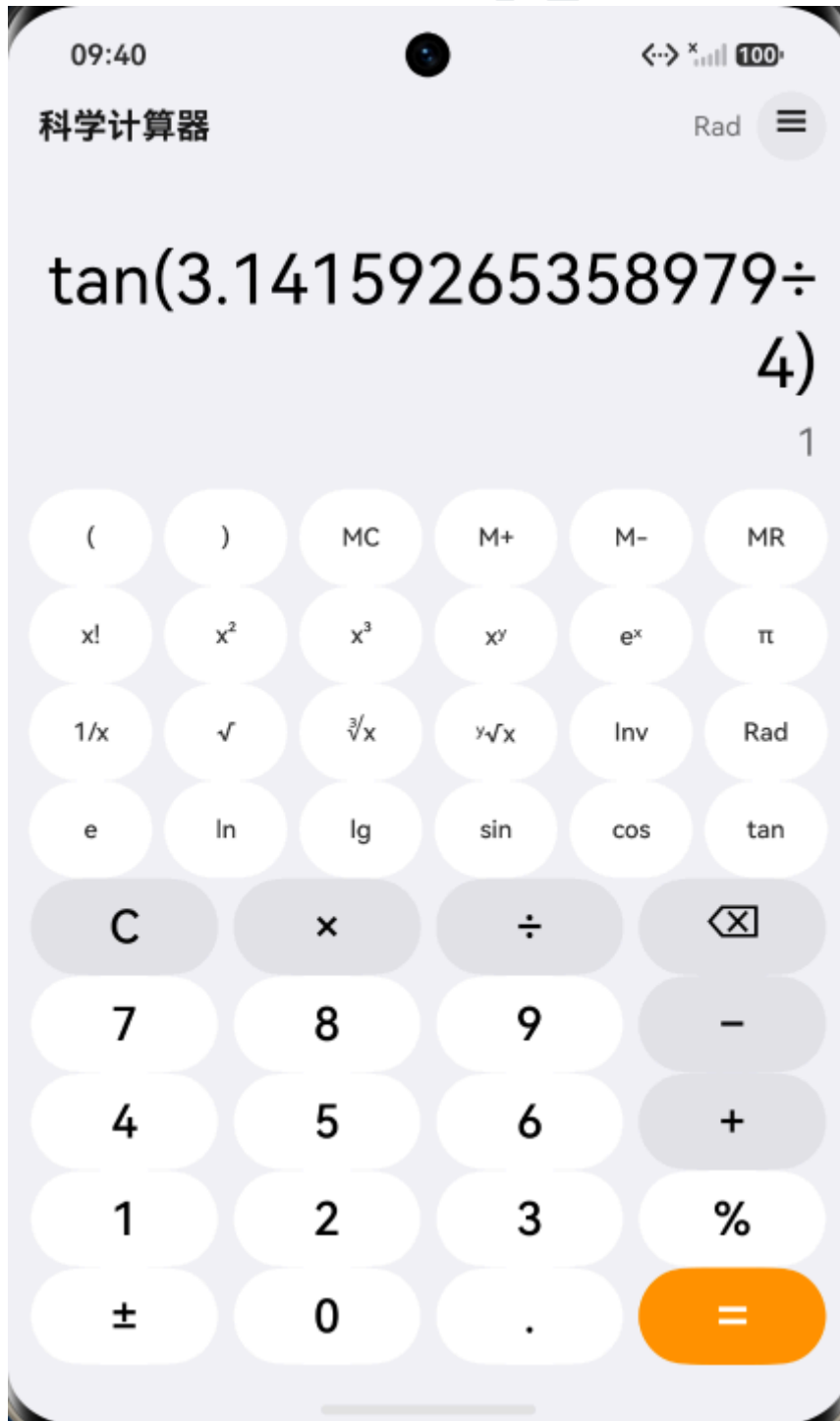
二、问题总结与体会

1. 实验中遇到的问题及解决方法

(1) **页面跳转不生效。** 最初 `router.pushUrl` 跳转亲戚称呼页无响应，原因是目标页必须注册在 `main_pages.json` 且标注 `@Entry`。解决方法是为每个页面加 `@Entry` 并写入注册表，同时改用 `router.replaceUrl` 保持单层页面栈。

(2) **科学计算器函数无法接收子表达式。** 最初的函数前缀 `pendingFunc` 只能吃单个数字，导致 `sin( $\pi/2$ )`、`ln(e)` 等输入不了常量或表达式。解决方法是把函数重构为一等 token（函数名 + `()`），并扩展调度场算法支持函数与一元负号，使参数可以是完整子表达式。

(3) 浮点噪声导致结果不干净。`tan( $\pi$ /4)` 会得到 0.9999999999999983。解决方法是在函数与幂运算返回前按 12 位有效数字舍入，得到 1、-1 等期望结果。



(4) 点右括号像点等号。闭合括号后 `updatePreview` 立即显示最终结果，使 `)` 与 `=` 视觉上无差别。解决方法是闭合括号后清空预览，只有 `=` 才真正求值。

(5) 非法关系输入影响后续操作。输入"爸爸的老公"会显示"—"且无法继续。解决方法是用 `tryAppendToken` 试算 token 追加是否合法，非法时静默忽略并保留原链。

2. 实验收获与体会

通过本次实验，我把华为官方的基础计算器示例扩展为一个包含六个功能的"多功能计算器"，对 HarmonyOS 的 ArkTS / ArkUI 声明式开发有了从页面搭建、状态管理到路由跳转、全屏适配的系统认识。